Projektni zadatak iz predmeta

# Informaciona Bezbednost

*Školska 2022/2023. Godina*

Projekat predstavlja veb aplikaciju za rad sa sertifikatima. Aplikacija se razvija kao monolit, u proizvoljnom steku tehnologija. Celokupno rešenje mora biti dostupno na nekoj platformi za kontrolu verzija. Sam projekat se radi u timovima od 3 studenta. Studenti ne moraju da budu iz iste grupe, samo će za odbranu kontrolnih tačaka ceo tim dolaziti u jednu grupu.

## Tipovi korisnika

- **Neautentifikovani korisnik** - Može da se registruje na sistem čime dobija običan nalog. Ako poseduje nalog, može da se prijavi na sistem.
- **Autentifikovani korisnik** - Korisnik koji se prijavio na običan profil. Ima mogućnost da pregleda i preuzme sve sertifikate. Takođe, može da zatraži da mu se izda sertifikat koji će biti potpisan od strane nekog drugog (*root/intermediate*) sertifikata. Može da zatraži da se određeni sertifikat povuče i može da izvrši validaciju bilo kog sertifikata preko ove aplikacije. Korisnik može da bude vlasnik samo *intermediate* ili *end* sertifikata. Može da odobri ili odbije zahteve za izdavanje sertifikata koji se odnose na izdavanje novih sertifikata potpisanih od strane onih koje posmatrani korisnik ima.
- **Admin** - Specijalni korisnik koji ne može da se registruje. Zadužen za upravljanje *root* sertifikatima koje koristi sama aplikacija kao i za odobravanje zahteva za izdavanje sertifikata koji su potpisane od strane *root* sertifikata. Može da uradi sve što i autentifikovani korisnik.

## Delovi sistema

- **Local file storage** - Mesto gde će se sami sertifikati čuvati
- **Baza podataka -** SQL/NoSQL baza podataka gde će se čuvati ostali neophodni podaci vezani za korisnike, vlasništvo nad sertifikatima…
- **Backend -** obavezn deo sistema gde će se implementirati logika, dozvoljeno je raditi u bilo kom jeziku i bilo kojoj tehnologiji.
- **Frontend -** nije obavezan i neće se ocenjivati. Pod ovim se smatra da mora postojati neki frontend za prikaz podataka ali se neće ocenjivati izgled, raspored komponenti i tome slično. U koliko tim želi, može koristiti neki *templating engine* za generisanje frontend-a na backendu poput Jinja2 za python, ASP.NET Core MVC za .NET ili Thymeleaf za javu.

## Funkcionalni zahtevi

1. **Registracija profila**
    a. Neregistrovani korisnik može da se registruje na sistem tako što će uneti validan email, ime, prezime, broj telefona, lozinku i dodatne podake, u koliko za njima ima potrebe. (2 boda)
    b. Nakon registracije korisnik mora da potvrdi vlasništvo nad makar jednim od unetih resursa (email ili broj telefona) time što će se poslati zahtev za potvrdu identiteta na taj resurs. (2 bod)
2. **Prijava na sistem**
    a. Neregistrovani korisnik može da se, unosom email-a i lozinke prijavi na sistem i tako pristupi funkcionalnostima autentifikovanog korisnika (2 boda)
    b. Neregistrovani korisnik tek nakon potvrde identiteta (email-a ili broja telefona) može da se prijavi na sistem i pristupi funkcionalnostima autentifikovanog korisnika (2 bod)
3. **Oporavak lozinke**
    a. Korisnik koji je zaboravio lozinku može da započne proces oporavka lozinke. Na resurs koji korisnik poseduje (email ili broj telefona) šalje se verifikacioni kod koji će korisnik uneti uz dva puta ponovljenu novu lozinku kako bi izvršio oporavak lozinke. (2 boda)
4. **Rotacija lozinki**
    a. Neophodno je implementirati mehnizam koji tera korisnike da nakon određenog vremena vrše obnavljanje lozinke. Za demonstraciju treba postaviti da lozinke mogu da važe proizvoljan period ali treba razmotriti i *best practice* za ovakav zahtev. Radi jednostavnosti dovoljno je da se pri svakoj prijavi proveri da li je od poslednjeg zanavljanja lozinke prošlo vreme koje je ta lozinka važila, i ako jeste naterati korisnika da lozinku zanovi kako bi mogao da nastavi interakciju sa sistemom. (1 bod)
    b. Treba omogućiti da korisnici ne mogu da zanove lozinku sa nekom od *n* prethodnih lozinki. (1 bod)

60 dana kod nas

zbog ovog treba voditi racuna o istoriji lozinki

5. **Pregled svih sertifikata**
    a. Svim prijavljenim korisnicima treba omogućiti pregled svih sertifikata. U sam pregled treba uključiti podatke o tome kad je sertifikat izdat, kome (kom korisniku) je izdat, tip sertifikata (*root, intermediate, end*). (1 bod)
6. **Preuzimanje sertifikata**
    a. Treba omogućiti korisniku da preuzme sertifikat. Ovo važi i za povučene i istekle sertifikate. (2 boda)
7. **Provera validnosti sertifikata**
    a. Omogućiti korisniku da proveri validnost nekog sertifikata na osnovu identifikatora. (2 boda)

b. Omogućiti korisniku da proveri validnost nekog sertifikata na osnovu upload-ovane kopije sertifikata. (2 boda)

8. **Pravljenje zahteva za novi sertifikat**
    a. Svaki autentifikovani korisnik ima mogućnost da napravi zahteva za izdavanje sertifikata. Korisnik može da zahteva samo *intermediate* ili *end* sertifikate, dok admin može da zahteva i *root* sertifikat. Zahtev se prosleđuje onom korisniku koji je vlasnik sertifikata koji treba da potpiše sertifikat za koji je podnet zahtev. Ako se sertifikat izdaje na osnovu sertifikata čiji je isti korisnik i vlasnik, zahtev se automatski odobrava. Ako se sertifikat izdaje na osnovu *root* sertifikata same aplikacije, admin mora da odobri sertifikat. Ako admin zahteva da se napravi neki sertifikat na osnovu nekog drugog sertifikata ili ako je to novi *root* sertifikat, zahtev se automatski odobrava. (3 boda)
9. **Pregled zahteva**
    a. Autentifikovani korisnik ima uvid u stanje aktivnih i prošlih zahteva za izdavanje sertifikata koje je podneo. (2 boda)
    b. Admin ima uvid u sve aktivne i prošle zahteve za izdavanje sertifikata. (1 bod)
10. **Odobravanje/odbijanje zahteva za sertifikat**
    a. Autentifikovani korisnik, koji je vlasnik sertifikata na osnovu kojeg postoji zahtev za izradu novog sertifikata, ima opciju da određeni zahtev prihvati, ili odbije. Ako prihvati pravi se novi sertifikat i potpisuje se izabranim sertifikatom. Ako odbije, iznosi razlog zbog kojeg se zahtev odbija i proces se završava. (3 boda)
11. **Povlačenje sertifikata**
    a. Autentifikovani korisnik koji je vlasnik nekog sertifikata (izuzev *root* sertifikata) može u bilo kom trenutku da povuče svoje sertifikate uz obrazloženje povlačenja istog. Obratiti pažnju na to da ako je sertifikat povučen, svi u lancu ispod njega se automatski povlače. To znači da svi sertifikati koje je on potpisao kao i sertifikati koje su ti sertifikati potpisali se automatski povlače. Admin može da povuče bilo koji sertifikat. (4 boda)
12. **HTTPS komunikacija**
    a. Sama aplikacija treba da koristi *root* sertifikat o kojem će admin voditi računa. Ako se frontend implementira kao zasebna aplikacija (popularni frontend radni okviri poput angular, vue.js ili react koji koriste odvojeni server) neophodno je da i oni koriste isti sertifikat. (1 bod)
    b. Baza podataka treba da koristi odvojeni sertifikat i treba omogućiti da baza i backend mogu da komuniciraju preko enkriptovanih poruka (u zavisnosti od tehnologije moguće je da protokol komunikacije između baze i servera

nije isključivo HTTP(s) tako da se mora omogućiti enkripcija poruka u protokolu koji tehnologija zahteva). (1 bod)

13. **Dvofaktorska autentifikacija**
    a. Kada se korisnik prijavljuje na sistem upotrebom lozinke, treba zatražiti da se potvrdi da je to zaista on tako što će se on dodatno autentifikovati preko verifikacionog koda koji će se poslati na jedan uređaj koji je korisnik izabrao (email ili telefon) (2 boda)
    b. Treba voditi računa o dužini trajanja sesije. Nakon isteka određenog vremenskog perioda treba naterati autentifikovane korisnike da se ponovo autentifikuju. Za demonstraciju je neophodno da ovaj period bude kratak, ali treba istražiti *best practice* metode. (1 bod)
14. **ReCAPTCHA za forme**
    a. Kako bismo zaštitili forme od “spamovanja” treba implementirati mehanizam ReCAPTHCA-e. Mogu se koristiti bilo koja rešenja za koja se tim dogovori (Google-ove ReCAPTCHAv2, ReCAPTCHAv3 ili Cloudflare Turnstile). Naglašava se da je validaciju tokena neophodno uraditi i na klijentskoj i serverskoj strani. (1 bod)
15. **OAuth protokol**
    a. Implementirati prijavu uz pomoć nekog delegiranog načina pristupa. Voditi računa da korisnik, ako ima registrovan nalog (email) i prijavi se uz pomoć OAuth protokola on ne treba da napravi novi nalog već da ga poveže sa postojećim. (2 boda)

## Nefunkcionalni zahtevi

1. **Validacija podataka (2 boda)** vezbe 4
    a. Sprečiti relevantne *Injection* napade
    b. Sprečiti *XSS* napade
    c. Sprečiti *Path Traversal* napade
    d. Izvršiti validaciju podataka, koristeći kriterijume validacije definisane po najboljim praksama za pisanje bezbednog koda
    e. Proveriti tipove datoteka koji se upload-uju (u slučaju validacije sertifikata putem uploadovanja), kao i ograničiti veličinu fajlova koji se mogu poslati.
2. **Zaštita podataka (2 boda)**
    a. Osetljivi podaci sa kojima aplikacija radi treba da budu obezbeđeni u skladištu, u transportu i tokom upotrebe. Identifikovati osetljive podatke i implementirati prikladne bezbednosne kontrole. Osetljivi podaci čije se skladištenje ne može izbeći treba da budu šifrovani ili heširani u zavisnosti od prirode podataka.
3. **Monitoring uz logove (2 boda)**

a. Kako log zapisi predstavljaju osnovni mehanizam za postizanje neporecivosti, neophodno je omogućiti njihovo prikupljanje. Od infrastrukture (operativni sistem, baza podataka, …) do alata (radni okvir, biblioteke), značajan deo koda nije pod našom kontrolom, ali to ne smanjuje našu odgovornost ako nam softver bude eksploatisan zbog ranjivosti u nekoj *third-party* komponenti te je neophodno voditi log zapise o tome ko pristupa kojim resursima na našem sistemu. Logove treba implementirati prateći *best practise* standarde. (Za projekat je dovoljno implementirati prikupljanje logova na nivou aplikacije, no studenti se podstiču da istraže kako bi mogli to da urade na nekom višem nivou, na primer na nivou sistema ili više distribuiranih sistema)

4. **Mailserver (2 boda)**
    a. Kako bismo izbegli probleme koje SMTP protokol može da donese, neophodno je implementirati integraciju sa nekim mail serverom koji će za nas da šalje poruke. Primer jednog takvog je Sendgrid, no danas ih ima dosta više.
5. **Ranjivosti (model pretnji) (2 boda)**
    a. Definisati alate koji će se koristiti za proveru različitih skupova komponenti (sqlmap, dependency-check…)
    b. Neophodno je izvršiti provere i napraviti neki vid izveštaja o tome šta je pronađeno i dokumentovati korake koji slede
    c. Analizirati ozbiljnosti ranjivosti i mogućnost eksploatacije
    d. Definisati i izvršiti strategiju za razrešavanje mogućih rizika

## Kontrolne tačke

U toku semestra će se održati dve kontrolne tačke nakon kojih sledi finalna odbrana. Same kontrolne tačke će se održati u terminu vežbi po rasporedu koji će naknadno biti objavljen i to:

1. U nedelji od 3.4.2023. do 5.4.2023.
2. U nedelji od 8.5.2023. do 10.5.2023.

Finalna odbrana će biti organizovana u junskom roku i biće subota prepodne (sadašnji termin koji je predložen je 17.6.2023. ali je moguće da će se ovaj datum malo pomeriti). Ako dođe do pomeranja termina, termin će biti pomeren za neki dalji datum u junu. Za dalje rokove kada će se projekat moći braniti bićete blagovremeno obavešteni i u tim rokovima će se bodovi skalirati.

Raspodela posla po kontrolnim tačkama:

- Prva kontrolna tačka (15 bodova):
    - 1a, 2a, 5a, 7a, 8a, 9a, 10a

    - Za ovu kontrolnu tačku ne morate imati nikakav frontend i dovoljno je da demonstrirate funkcionalnost kroz Postman.
    - Takođe, za ovu kontrolnu tačku nije neophodno voditi računa o *best practise* predlozima za kontrole.
- Druga kontrolna tačka (15 bodova):
    - 1b, 2b, 3a, 6a, 7b, 9b, 11a
    - Za ovu kontrolnu tačku je neophodno započeti frontend, i očekuje se da će se minimalno (ili nikako) koristiti Postman.
    - Ovde će se pregledati i *best practise* predlozi za stvari koje su do sad implementirane
- Finalna odbrana (20 bodova):
    - 4a, 4b, 12a, 12b, 13a, 13b, 14a, 15a
    - Očekuje se da na ovoj proveri ne koristite Postman za demonstraciju, već da se sve izvršava kroz grafički interfejs
    - Na ovoj proveri će se pregledati poštovanje *best practise* predloga kao i nefunkcionalni zahtevi
        - Napomena za nefunkcionalne zahteve: Iako se sastoje iz tačaka, bodovanje će biti rađeno po principu “ili sve ili ništa”. Tako na primer da biste dobili sve bodove vezane za tačku *Validacija podataka* morate ispoštovati sve tačke od *a* do *e*.

## Bodovi

Projekat nosi 50 poena, i ostalih 50 poena nosi usmeni ispit. Za polaganje predmeta neophodno je da imate 50% bod na oba dela i da je ukupan broj poena >= 51.

Važi pravilo da na jednoj odbrani možete vratiti bodove samo sa prethodne odbrane što znači da ako ste izgubili neke bodove na jednoj kontrolnoj tački možete ih vratiti samo na proveri koja sledi (bilo da je kontrolna tačka ili finalna odbrana). Izuzetak su odbrane projekta u kasnijim rokovima gde nema povraćaja izgubljenih bodova sa kontrolnih tačaka.

## Napomena

Iako se broj bodova na odbrani u daljim rokovima skalira i dalje morate ostvariti minimalan broj bodova iz obe oblasti kako biste mogli da položite predmet što znači da će vam biti neophodan veći broj implementiranih stvari za isti broj bodova.

## FAQ

**Q:** Da li vama treba da prosledimo link i do repozitorijuma za frontend ili samo za backend?

- U principu kako god odlučite. Možete dodati više polja u excel za prijavu timova.

**Q:** Kako ke zamisljeno cuvanje sertifikata u local file storage? Jel to samo obican txt fajl i onda kad trazimo sertifikate prolazimo kroz svaki? Ili pravimo neku slozeniju strukturu?

- Postoje određene strukture za čuvanje sertifikata na lokalnom disku, konkretno svaka tehnologija ima način za izvoz u ove tipove datoteka (.crt + .key/, ili .pfx). No na trećim vežbama je spominjan i *keystore*

**Q:** Razlika izmedju preuzimanja i pregleda sertifikata?

- Pod pregledanje sertifikata je zamišljen da bude prikaz na veb stranici, u nekoj tabeli, a preuzimanje bi bilo preuzimanje kopije sertifikata sa servera

**Q:** Sta se misli pod moze da izvrsi validaciju bilo kod sertifikata?

- Da postoji endpoint na koji može da se pošalje SERIJSKI BROJ sertifikata i da se kao odgovor dobije da li je sertifikat validan ili ne, pogledajte OCSP (Online Certificate Status Protocol) sa trećih vežbi. Podsećam sertifikat je validan ako nije istekao period važenja sertifikata i ako sertifikat, ili neki u lancu iznad njega, nije povučen.

**Q:** Koji su dodatni podaci potrebni za cuvanje korisnika, da li u to spada jmbg ili nesto tako

- Sami procenite, kao što smo rekli, svaki podatak koji čuvate morate da adekvatno zaštitite, a najbolje zaštićeni podaci su oni koji se ne čuvaju. Trudite se da čuvate samo one podatke koji su neophodni za rad samog servisa

**Q:** 2b da li mi biramo na koji ce nacin korisnik da potvrdi identitet prilikom logina ili mu saljemo i poruku i mejl?

- Trebalo bi korisnik da bira, a vi da implementirate oba načina. Možete videti da li postoji neki api koji može da šalje poruke na WhatsApp/Viber, ne mora biti SMS poruka.

**Q:** 4a da li ovo znaci da npr na svakih nedelju dana neki zahtev  se kreira za odredjenog kornisnika i cuva koji govori da je vreme da se sifra promeni? 4b gde cuvamo te stare sifre, jel to sve u bazi i koliko je to pametno da teramo korisnika da menja sifru odredjeni period?

- Tako je, sa tim što možete to implementirati na jednostavniji način. Na primer, čuvate kada je korisnik poslednji put promenio lozinku i ako je, prilikom sledeće prijave, prošlo više od n vremena, nećete mu dati da se prijavi već ćete ga naterati da zanovi lozinku. Na vama je da istražite i smislite gde ćete čuvati stare lozinke i kako. Što se tiče toga koliko je pametno, ja bih rekao da je to neophodno za visok stepen bezbednosti.

**Q:** 6a Da li korisnik moze povuci i tudje sertifikate? I sta spada pod povlacenjem sertifikata

- Ne, korisnik može povući samo svoje sertifikate, izuzev admina koji može povući sve sertifikate. Za to šta spada u povlačenje sertifikata pogledajte 3. Vežbe

**Q:** 7b sta je uploadavana kopija sertifikata?

- Imate polje gde možete da unesete vašu lokalnu kopiju sertifikata, server pročita taj sertifikat i vidi da li ima u bazi podatak o tom sertifikatu i ako ima proveri da li je validan ili ne. Ako nema kaže da nije validan. Funkcionalnost slična kao i 7a samo što ćete serijski broj pročitati iz uploadovanog sertifikata a ne kao neki parametar zahteva.

**Q:** Nefunkc 3. Jel za te logove znaci kad god neko napravi zahtev mi uz pomoc nekog aspekta prvo upisemo taj zahtev u log i onda dalje vrsimo sam zahtev(kao sto se jwt proverava pre svakog zahteva tako i ovo pre jwta?)

- Ne baš, radićemo uskoro neoporecivost uz logove, ali log zapis samo predstavlja zapis da se nešto na sistemu desilo. Log nije uslov niti preduslov ni za šta već predstavlja stavku koja, ako postoji, garantuje da se ono što u njoj piše dogodilo. Ideja je da možete da rekonstruišete (ne 100%) događaje koji se dešavaju ako dođe do otkaza sistema.

**Q:** Je l delove sertifikata cuvamo u bazi ili sve cuvamo preko keystore koji ste nam ostavili u vezbama

- Tehnički ja vam ne bih trebao reći odgovor na to pitanje, imate opciju da kad ja zatražim da vidim sve sertifikate, vi prođete kroz svaki mogući sertifikat izvučete podatke i ispišete (što bi bilo čak i sporo jer treba vreme da se pročita svaki fajl), ili da neke informacije čuvate u bazi što će vam dosta olakšati. Koje informacije, to je na vama.

**Q:** Koji tip fajla koristi keystore i na koji nacin radi?

- Ako mislite koje ekstenzije poznaje keystore, poznaje mnoge i praktično nemate "jedan koji svi koriste", neki kao npr .pfx zajedno čuvaju javni i privatni ključ (koji je encryptovan) dok neki drugi ih čuvaju odvojeno. Dodatno to što ste generisali sertifikat i sačuvali ga u jednom fromatu ne znači da ne možete da ga sačuvate u nekom drugom. Za to vam savetujem da koristite npr java keytool. Vi možete razne tipove uvesti u keystore i on će ih prepoznati. Na vežbama nismo prelazili primer koji radi sa keystore kroz kod, pa neće biti obavezno, ali bilo bi lepo da znate da opcija postoji. (Uvod u keytool)

**Q:** Da li mi samo extendujemo root sertifikat koji je dat preko x509 ili pravimo svoje intermediate i end sertifikate od 0?

- Poslednje pitanje nisam najbolje razumeo ali hajde da ugrubo nabrojim opet zamišljen proces rada. Prvo generišete jedan root sertifikat (self-signed certificate) i taj sertifikat koristi i sam servis za HTTPS (koji je za dalje tačke). Na osnovu tog root sertifikata vi možete izdati nove sertifikate (i intermediate i end sertifikate). Kad ste dobili neke intermediate sertifikate, njih možete koristiti da dalje izdate nove (intermedaite i end sertifikate).

**Q:** Na primeru sa vežbi primetim da su data dva načina za učitavanje sertifikata, jedan koristi KeyStore klasu, a jedan čita direktno iz fajla. Zamolio bih za pojašnjenje razlike između ova dva načina.

- Rekli smo da čuvanje sertifikata može da se radi na više načina. Jedan je da se čuva direktno na fajl sistemu, samim tim to su neki obični fajlovi sa ekstenzijama .pem/.cert/.key./pfx ... kojim god. Mana toga je što ako neko pristupi našem serveru on će moći bez ikakvih problema da dobavi sve te fajlove. Malo bolje je da koristimo keystore. Keystore je kao mali file storage za čuvanje tajni na sistemu. Sadržaj je enkriptovan, ali ne ceo, na primer vi i bez lozinke možete dobiti javne delove svega, dok vam treba lozinka da biste dekriptovali privatne ključeve. Kada koristite key store dobijete fajlove sa ekstenzijom jks načešće (kao java keystore) ali su i ostale tehnologije prihvatile ovaj pristup. Samim tim kako god želite, nama je neophodno da imamo one feature koje smo pobrojali i da vi znate da argumentujete prednosti i mane onoga što ste izabrali. Nismo eksplicitno tražili keystore. U jednom keystore-u možete čuvati više sertifikata i ključeva, tako da zamislite da bi samo admin imao lozinku od keystore i npr sam api bi imao to kao neku tajnu i mogli bi to da koriste, ako se odlučite za taj pristup

**Q:** Prilikom čitanja privatnog ključa korišćenjem KeyStore klase potrebna nam je lozinka. Šta tačno predstavlja ta lozinka. Da li je ta lozinka nešto što korisnik treba da odredi prilikom pravljenja sertifikata ili je vezana isključivo za server.

- Ovo sam objasnio u prethodnom pitanju, ali da, Javni delovi nisu enkriptovani jer i nisu tajne, tajni delovi su enkriptovani i zato nam je neophodna lozinka. Tehnički vi možete da imate više keystore-ova na sistemu gde je svaki keystore vezan za korisnika ali će vam ovo otežati rad i nije trenutno poenta.

**Q:** Ako koristimo KeyStore klasu, za sertifikate se takođe vezuje i alijas. Da li ima smisla da taj alijas bude id sertifikata u bazi?

- Alijas se vezuje i za sertifikate konkretno. Idealno treba da bude nešto što je jedinstveno i što bliže opisuje sertifikat. Ne mora da bude jedan podatak, može biti kombinacija više. Npr ime-prezime-serijski_broj mi deluje kao okej alijas. Mada može i samo serijski broj.

**Q:** Kada pravimo sertifikat potrebno je da navedemo podatke kao što su: name, givenname, common name, oragnizacija, zemlja, organization unit name, email itd. Da li to znači da sve ove podatke treba da tražimo od korisnika prilikom registracije i kada taj korisnik odluči da napravi
sertifikat uvek se koriste podaci koje je dao prilikom registracije ili bi možda trebalo da prilikom svakog kreiranja sertifikata tražimo od korisnika da unese te podatke.

- Što se toga tiče kako želite. Meni ima smisla da se neki podaci čitaju iz baze a neki traže od korisnika svaki put. Npr to što ste pobrojali ima smisla da se čita iz baze, dok npr, organizacija može da se menja, može svakako da se menja i tip sertifikata (intermediate, end) i možda još nešto ako vam padne na pamet.

**Q:** Sto se tice autorizacije, da li ste predvideli da koristimo JWT token ili predlazete neki drugi pristup koji bi bio preferiran?

- Imate kompletnu slobodu, treba da bude neki način i da obezbedite best practise za tu metodu. Da li koristite JWT ili npr ASP.NET Core Identity sa cookie mehanizmima na vama je skroz.

**Q:** 12.b nam nije najjasnija, da li to znaci da mozemo da koristimo in memory H2 sa Springom, ili je ipak potrebno da to bude neka distribuirana instanca (ne znam da li je to pravi naziv, ali ne-in-memory instanca)? Ako biste mogli pored odgovora dodatno da pojasnite ovu tacku.

- Baza podataka je odvojen servis (najcesce je to neki server koji nudi razne protokole, među njima nudi i http protokol). Samim tim vi kad je "instalirate" vi ste zapravo na vašoj mašini pokrenuli lokalni server (ako koristite Windows možete stisnuti "window" i kucati "Services" i tu ćete naći postgresql server da trči kao servis). Taj server ima neki setup folder koji, ako niste ništa podešavali dok ste instalirali server ima default podešavanja. Vi treba da nađete kako da dodate sertifikat u tu lokalnu instancu servera i da vaš server koristi taj sertifikat.
- Distribuiranost baza je samo mogućnost da imate servis nekako (logički ili fizički) podeljen na više mašina. Neke baze nude mogućnost clusterovanja kako biste mogli da imate high availability i scalability ako neka instanca ne radi (jer ako vam recimo padne baza vaša aplikacija neće raditi iako je sam kod dobar, o ovome ćete verovatno slušati kod Eve na cloudu). To nije jedini slučaj jer imate i sharding i mnoge druge stvari. Ako vas zanima više možemo prodiskutovati na vežbama jer je opširnija tema ovo je samo neko kratko objašnjenje.
- Problem kod H2 je što to nije servis za sebe, to je samo dodatno parče koda koje trči na istom serveru kao i sama aplikacija (to znate tako što vi ne menjate "url" kada pričate sa bazom, jer oba trče na npr localhost:8080, a ako hoćete da pristupite bazi najčešće bude samo još jedan endpoint na istom servisu npr localhost:8080/h2, dok kod postgresa npr, vaš api trči na localhost:8080, a postgres najčešče na localhost:5432, i sama razlika u portovima vam govori da su to odvojeni servisi jer ne mogu dva servisa da rade na istom portu na istoj mašini).

**Q:** Gde cuvamo privatne a gde javne kljuceve, jel isto u key store kao i sertifikate?

- Kako želite, možete da čuvate na fajlsistemu u razdvojenim folderima (npr da imate *certs* folder za sertifikate, i da imate *keys* folder za čuvanje ključeva). Ako koristite keyStore moj je savet da sve čuvate u keyStore, i sertifikate i privatne ključeve.

**Q:** Za povlacenje sertifikata, da li treba u konkretnim fajlovima na sistemu nesto da se menja, ili je dovoljno u bazi samo promeniti status u nevalidne?

- To pitanje je dosta široko ali probaću ugrubo da odgovorim. Pre svega to zavisi od izabrane metode provere povučenosti sertifikata. Pošto smo rekli da imamo dve metode (CRL i OCSP) tehnički imate dve mogućnosti ali smo rekli da ćemo mi raditi OCSP pa ću objasniti kako bi to išlo otprilike (u stvarnom sistemu):
    1. Učitate sertifikat i preko njega nađete njegovog issuera.
    2. Issueru tog sertifikata (tom sertifikacionom telu, eng CA) pošaljete zahtev za povlačenje i to je to. Vodite računa da postoji mogućnost da se sertifikati keširaju tako da je najbolje posle ovoga uraditi "redeploy" sertifikata.

3. (Opciono) odgovor od issuer-a se najčešće sačuva (nekad kao fajl na disku) kao neki vid dokaza da je sertifikat povučen baš zbog toga što se većina sertifikata kešira i treba neko vreme da se oni svuda propagiraju i status postane "živ"

- E sad, pošto mi nemamo baš mogućnost da šaljemo nekom drugom sertifikacionom telu zahtev (mi smo CA za naš projekat) mi bismo samo kod sebe u bazi promenili neki flag ili enum kao što ste i rekli. Vodite računa da mi je bitno da imamo razlog povlačenja kada se sertifikat povlači i da povučeni sertifikat ne može da radi ništa (ne može opet da se povuče ili izdaje sertifikate).

**Q:** implementirao sam verifikaciju putem mail-a kao i slanje verifikacionog koda putem sms-a. Zanima me da li prilikom registracije saljemo i mejl i sms zajedno. Ukoliko saljemo i jedno i drugo, a korisnik nije uneo
ispravan broj telefona, da li se korisnik svejedno pravi i cuva u bazi samo kao neverifikovan i da li idalje ima mogucnost da se verifikuje putem mejla iako je mobilni nevalidan i nije moguce poslati mu sms. Isto pitanje i oko reset-a lozinke da li i tu saljemo na oba izvora odjednom? Ovo pitanje imam zato sto Twilio koji sam koristio za slanje sms poruka odmah izbaci eror ukoliko je broj nevalidan i onda mi nema smisla da response registracije bude 404 zbog nevalidnog broja, a korisnik se svejednonapravi u bazi. Takodje posto koristim besplatni Trial za Twilio da li jeproblem sto nemam mogucnost slanja poruka apsolutno svim brojevima telefona?

- Ideja je bila da vi korisniku ponudite oba a on da izabere jedan. Zamislite to kao neki drop down iz kog bi on mogao da izabere metodu kojom ce izvršiti dvofaktorsku autentifikaciju ili bilo koju drugu opciju. Ako biste hteli da podržite nešto što ste predložili to se onda zove multifaktorska autentifikacija i ona iziskuje da se sve izvrši potvrdno (i sms i email). Za ovaj projekat nije neophodno tako nešto ali bi svakako poboljšalo bezbednost.
- Što se response code-ova tiče, savetujem da ne vraćate 404 na to pošto je to "not found" više bi imalo smisla da bude recimo 400 što je "bad request"
- Nije problem što nisu svi brojevi telefona